Private Github Actions without PAT

Rohit Goswami · 2020-12-23 · tools · github · workflow

> *A workflow for managing private submodules in a private repository without personal access tokens for Github actions*

Background

Ever since Travis CI decided to drink corporate kool-aid, the search for a new CI has been brought to the fore again. Github Actions seem uniquely suited for private repositories, since most CIs bill for private repos. However, the basic authentication setup for the checkout action involves using one SSH key, effectively a personal access token, for both the main project and all submodules. This is untenable for anyone working with a team.

Solution

The fix, as it were, is in two steps. We will first require a deploy key to be set for the private submodule, and then store the private portion in the private repo. We will begin with a concrete setup.

Setup

Consider a standard C++ build setup:

```
name: Fake secret project

on:
  push:
    branches: [master, development]
  pull_request:
    branches: [master]

jobs:
  build:
    runs-on: ${{ matrix.os }}

    strategy:
      max-parallel: 4
      matrix:
        os: [ubuntu-18.04]
        cpp-version: [g++-7, g++-9, clang++]

    steps:
      - uses: actions/checkout@v2
      - name: build
        env:
          CXX: ${{ matrix.cpp-version }}
        run: |
          mkdir build && cd build
          cmake -DCMAKE_CXX_COMPILER="$CXX" -DCMAKE_CXX_FLAGS="-std=c++11" ../
          make -j$(nproc)
      - name: run
        run: |
          ./super_secret
```

Key Generation

This section is standard. We will generate a deploy key essentially. Note that it isn't necessary to set a password, using one would only minimally improve security and bring in some annoying script modifications.

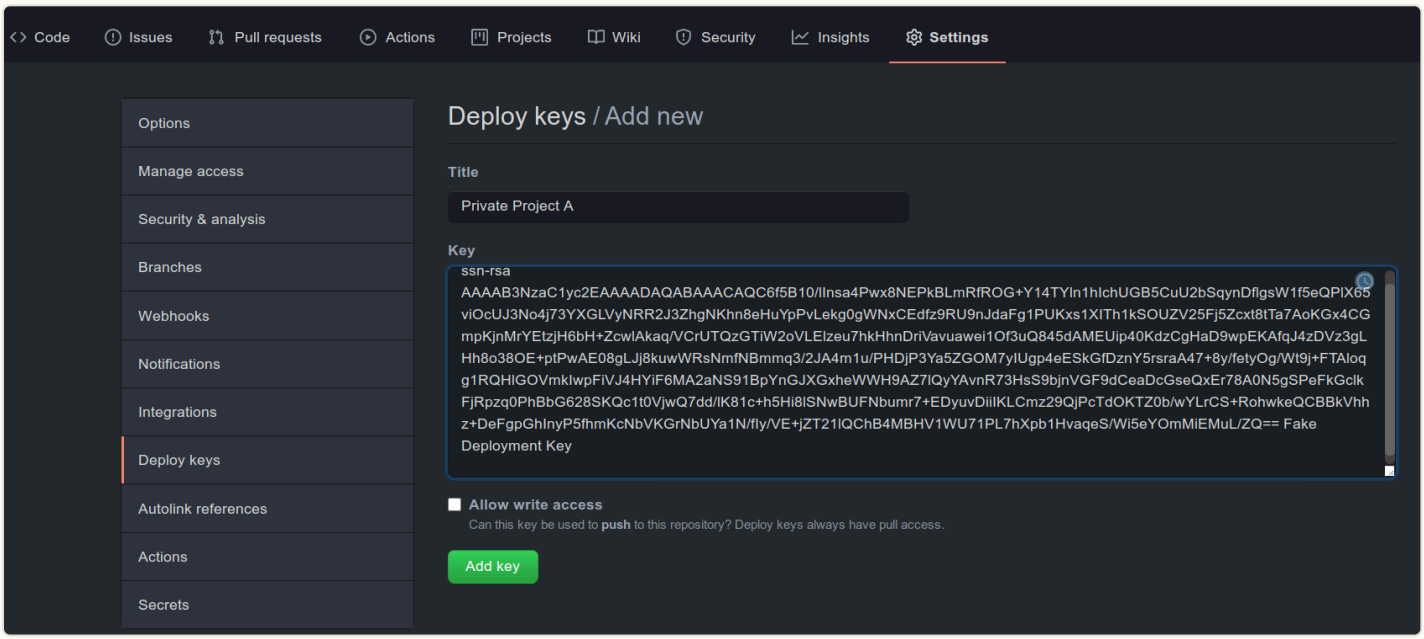
```
# Anywhere safe
ssh-keygen -t rsa -b 4096 -C "Fake Deployment Key" -f 'priv_sub_a' -N ''
```

Private Submodule Repo

We will store the **public portion of the key** as a deploy key in the **private submodule repository**.

```
# Copy contents
cat priv_sub_a.pub | xclip -selection clipboard
# For wayland
cat priv_sub_a.pub | wl-copy
```

Note that, as shown in Fig. 1 we do not need to give write access to this key.



Private Project Repo

Now we will need the **private portion of the key** as a secret in the **private project repository** (see Fig.).

```
# Copy contents of private key
cat priv_sub_a | xclip -selection clipboard
```

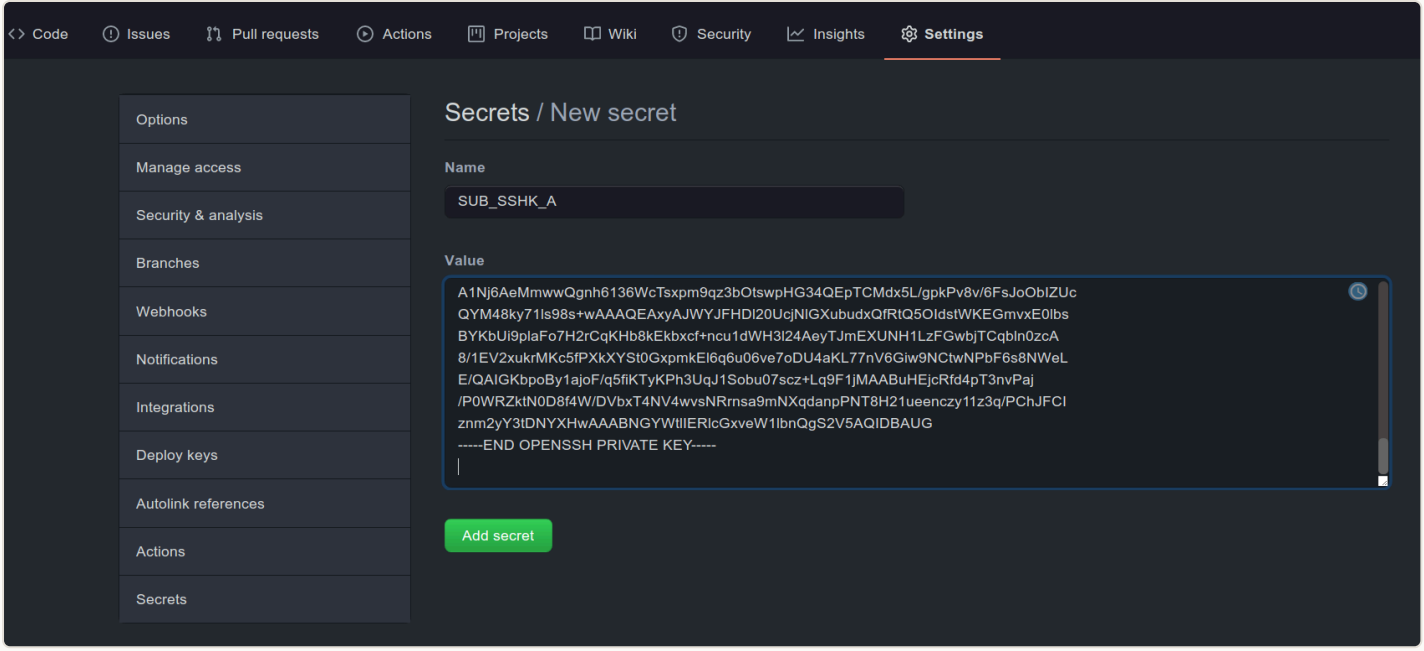


Figure 1: Secret setup in the private project

Workflow Modifications

Now we will simply update our workflow [^fn:1]. We will simply add the following step:

```
- name: get_subm
  env:
    SSHK: ${{ secrets.SUB_SSHK_A }}
  run: |
    mkdir -p $HOME/.ssh
    echo "$SSHK" > $HOME/.ssh/ssh.key
    chmod 600 $HOME/.ssh/ssh.key
    export GIT_SSH_COMMAND="ssh -i $HOME/.ssh/ssh.key"
    git submodule update --init --recursive
```

This will work for a single private submodule and multiple public submodules. For multiple private submodules, we would not initialize them recursively, but instead use a separate key for each.

```
- name: get_subm_a
  env:
    SSHK: ${{ secrets.SUB_SSHK_A }}
  run: |
    mkdir -p $HOME/.ssh
    echo "$SSHK" > $HOME/.ssh/ssh.key
    chmod 600 $HOME/.ssh/ssh.key
    export GIT_SSH_COMMAND="ssh -i $HOME/.ssh/ssh.key"
    git submodule update --init -- <specific relative path to submodule A>
- name: get_subm_b
  env:
    SSHK: ${{ secrets.SUB_SSHK_B }}
  run: |
    mkdir -p $HOME/.ssh
    echo "$SSHK" > $HOME/.ssh/ssh.key
    chmod 600 $HOME/.ssh/ssh.key
    export GIT_SSH_COMMAND="ssh -i $HOME/.ssh/ssh.key"
    git submodule update --init -- <specific relative path to submodule B>
```

Note that it is not possible to use the same SSH key in multiple submodule repositories, as each deploy key can only be associated with one repository.

Complete Workflow

Putting the above steps together, for the case of a single private submodule and multiple public submodules, we have:

```
name: Fake secret project

on:
  push:
    branches: [master, development]
  pull_request:
    branches: [master]

jobs:
  build:
    runs-on: ${{ matrix.os }}

    strategy:
      max-parallel: 4
      matrix:
        os: [ubuntu-18.04]
        cpp-version: [g++-7, g++-9, clang++]

    steps:
      - uses: actions/checkout@v2
      - name: get_subm
        env:
          SSHK: ${{ secrets.SUB_SSHK_A }}
        run: |
          mkdir -p $HOME/.ssh
          echo "$SSHK" > $HOME/.ssh/ssh.key
          chmod 600 $HOME/.ssh/ssh.key
          export GIT_SSH_COMMAND="ssh -i $HOME/.ssh/ssh.key"
          git submodule update --init --recursive
      - name: build
        env:
          CXX: ${{ matrix.cpp-version }}
        run: |
          mkdir build && cd build
          cmake -DCMAKE_CXX_COMPILER="$CXX" -DCMAKE_CXX_FLAGS="-std=c++11" ../
          make -j$(nproc)
      - name: run
        run: |
          ./super_secret
```

Alternative

Another way to handle this with less boiler-plate for cloning a private repository is to use the `webfactory/ssh-agent` action.

```
- name: Setup SSH for private submodule
  uses: webfactory/ssh-agent@v0.9.0
  with:
    ssh-private-key: ${{ secrets.SUBMODULE_PRIVATE }}
- name: Pull GPRD submodule
  run: |
    git clone git@github.com:TheochemUI/gpr_optim.git subprojects/gpr_optim
    cd subprojects/gpr_optim
    git checkout e2ac
```

Conclusions

We have demonstrated a minimally invasive setup for working with a private submodule, which is trivially extensible to multiple such submodules. With this, it appears that GH actions might a viable option (as opposed to say, Wercker), at least for private teams. A more full comparative post might be warranted at a later date.
